

全系统模拟器访存慢路径自适应优化

Adaptive Memory Slow-Path Optimization for Full-System Emulators

摘要

针对现代全系统动态二进制翻译器中软件地址转换慢路径成本高、无条件增加缓存查询可能产生性能回退的问题，提出大页转换缓存、非叶页表缓存及在线收益率门控。两类缓存仅在主 SoftTLB 与 victim 均未命中后工作，门控依据大页命中和跳过页表层数在采样、启用和旁路间切换。以 QEMU 8.2.9 x86-64 系统模式为对象，分离无插桩计时、性能采样、机制计数和客户机访存分母。固定 4096 项并关闭 victim 时，429.mcf 的自适应组合取得 1.100 倍配对加速；默认动态配置下中位为 0.998 倍，未显示可辨收益。4 KiB 负对照中，门控旁路 85.7% 的候选查询，并将波动恢复到接近基线。结果表明，miss-only 缓存可利用有限转换覆盖后的剩余局部性，自适应策略能抑制明显低复用区间的附加开销，但不能消除地址布局和宿主调度造成的长尾。

关键词： 全系统模拟；动态二进制翻译；软件转换后备缓冲器；大页；页表遍历；自适应缓存；地址转换

Abstract

Software address-translation slow paths remain costly in full-system dynamic binary translators, while unconditional cache probes can regress workloads with little reuse. This paper presents a large-page translation cache, a non-leaf page-table cache, and a benefit controller. Both caches are consulted only after the primary SoftTLB and victim cache miss; the controller alternates among sampling, active, and bypass phases using large-page hits or avoided walk levels as proxies. The QEMU 8.2.9 x86-64 implementation separates uninstrumented timing, performance sampling, mechanism counters, and guest-memory denominators. With a fixed 4096-entry SoftTLB and no victim cache, the adaptive combination achieves a 1.100x paired median speedup on SPEC CPU2006 429.mcf. With QEMU's dynamic SoftTLB and victim cache, its paired median is 0.998x and not distinguishable from baseline. In a 4 KiB negative control, the controller bypasses 85.7% of probes and restores variability close to baseline. Thus miss-only caches expose residual translation locality, while adaptation limits, but cannot eliminate, workload-sensitive regressions.

Keywords: full-system emulation; dynamic binary translation; software TLB; large page; page-table walk; adaptive caching; address translation

1 引言

全系统动态二进制翻译需要模拟客户机处理器、内存管理单元和设备，并将每次客户机访存转换为安全的宿主访问。QEMU[4] 在生成代码中内联软件转换后备缓冲器（software translation lookaside buffer, SoftTLB）查询，使常见命中只执行索引、标签比较和地址修正；未命中则进入目标架构页表遍历、地址空间解析和表项安装等慢路径。Tong 等在早期 QEMU 上测得访存模拟平均占执行时间的 38.1%，并通过容量、关联度、大页和填充优化获得 24.4% 的平均加速[1]。此后 QEMU 引入动态伸缩主表和全相联牺牲 SoftTLB，早期瓶颈不能直接外推。近年的 Victima、Utopia 等工作利用缓存资源或受限映射扩大硬件地址转换覆盖范围[8-9]，Ninja 针对嵌套翻译减少多阶段遍历[10]，相关工作负载研究也表明地址转换开销随访问模式和并发规模显著变化[11]。这些机制主要面向硬件。面向跨 ISA 模拟，HSPT 借助宿主页表和 TLB，BTMMU 以 Dual-TLB 和硬件异常缩短公共转换路径[12-13]；影子映射缺失后仍需由软件遍历客户机页表并建

立映射。对纯软件模拟器而言，新增查询若进入每次访存，会增加宿主指令和缓存压力。为此，本文只优化主表和牺牲表均未命中的路径，设计大页转换缓存和非叶页表缓存，再按运行期复用收益自适应旁路低效查询。实验同时覆盖 QEMU 默认动态 SoftTLB 和 固定容量压力配置：前者用于判断现代软件机制后的剩余空间，后者作为有限硬件转换覆盖的受控代理。结果显示，定向微基准和固定容量的 429.mcf 存在明确收益，默认动态配置则基本持平，自适应门控只能限制而不能保证消除回退。本文的贡献包括：（1）在现代 QEMU 的 miss-only 路径实现大页转换复用和非叶页表复用，并区分嵌套翻译中的失效覆盖范围与线性重建范围；（2）设计无需硬件计数器的分窗自适应门控，用可避免的 fill 和页表访问识别低收益区间，同时保持生成代码命中路径不变；（3）建立计时、性能采样、机制计数与客户机访存分母相分离的验证方法，在固定容量代理、QEMU 默认动态容量及无复用负对照中给出收益和边界，并讨论其对 HSPT、BTMMU 等硬件辅助跨架构模拟剩余慢路径的适用性。

2 访存通路及开销测量

2.1 QEMU 系统模式访存通路

QEMU 的翻译代码块在执行客户机 load/store 时首先查询当前内存管理模式对应的 SoftTLB。命中项包含标签、权限和宿主地址修正量，可直接形成宿主访问。主表未命中时，运行时函数先查询 8 项全相联牺牲表；再次未命中后调用目标架构 TLB fill。对 x86-64 客户机，fill 可能执行四级或五级页表遍历，也可能在嵌套虚拟化中继续遍历第二阶段页表。得到客户机物理地址后，QEMU 解析 MemoryRegion，并由 `tlb_set_page_full()` 安装普通页表项。图 1 给出通路及本文缓存与门控的位置。

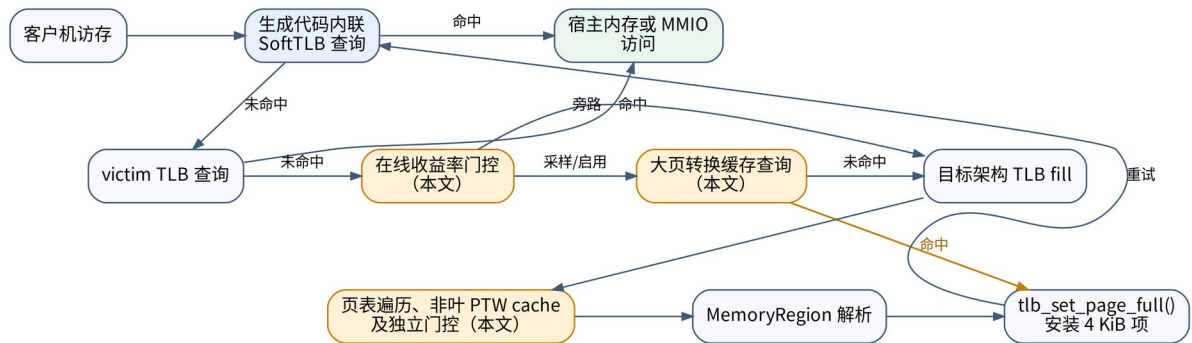


图 1 QEMU SoftTLB 访存通路及优化位置

大多数访存停留在图 1 上方的内联命中路径。若在该路径增加一次哈希、分支或额外访存，其成本会被所有客户机内存操作放大。本文因而把新增结构放在两级 SoftTLB 均未命中之后：大页缓存位于目标架构 fill 之前，页表缓存位于 x86 walker 内部。

2.2 分离式测量方法

本文采用四类相互分离的运行。无插桩 timing 构建用于端到端时间，且关闭 QEMU JIT perf map，避免翻译代码映射文件带来的磁盘写入；同一无插桩构建配合 perf 采样用于函数级时间归因；profile 构建记录主表未命中、牺牲表命中、refill、页表遍历和缓存事件；TCG plugin 在客户机标记区间内统计 load/store，提供事件率的精确分母。插桩运行不参与加速比计算。

对除原子特殊路径外的访问，事件计数应满足

$$N_{\text{refill}} = N_{\text{L1 miss}} - N_{\text{victim hit}}$$

分析器在 origin、读写类型和内存管理模式三个维度检查该关系，并验证客户机标记、远端返回码、二进制哈希和运行参数。全部计数记录中实际出现的 134 个 origin—访问类型—内存管理模式组合，其计数残差均为 0。

2.3 慢路径画像

表 1 给出六个应用负载的 SoftTLB 事件率与宿主周期占比。named SoftMMU 表示能够归因到慢路径符号的周期，不包含生成代码内联的命中查询；事件率来自独立 plugin 和 profile 运行。

表 1 QEMU SoftTLB 慢路径开销

工作负载	named SoftMMU	data refill/访存	victim 恢复率
sysbench random 4 KiB write	5.21%	0.0587%	44.46%
stress-ng tlb-shootdown	16.84%	0.4948%	19.14%
DaCapo avrora	6.61%	0.0372%	83.15%
nested QEMU/KVM random	13.87%	0.1276%	85.13%
SPEC CPU2006 429.mcf train	15.22%	0.2649%	8.95%
GAPBS PageRank scale-20	1.17%	0.0246%	88.00%

六个负载的 data refill 均低于客户机访存的 0.5%，但可命名慢路径最高占宿主周期 16.84%，说明慢路径具有低频、高单次成本特征。429.mcf 中 91.05% 的 data L1 未命中最终进入 refill，平均每次页表遍历访问 4.19 层，是最明显的应用级转换压力负载；GAPBS 中 88.00% 的 L1 未命中被牺牲表挽回，稳定阶段的优化空间较小。

为排除动态扩容和牺牲表的遮蔽效应，本文进一步将每个内存管理模式的主表固定为 4096 项并关闭牺牲表。在该受控配置下，六个负载的 named SoftMMU 平均占比为 13.39%，429.mcf 最高为 24.18%。这一数值仍低于文献[1]的完整访存模拟占比，主要原因是内联命中查询无法从客户机有效计算中单独归因，因此 13.39% 是可命名慢路径的下界。

2.4 硬件辅助跨架构模拟中的慢路径

HSPT 将客户机地址空间映射到宿主页表，使常见访问利用宿主 MMU 和 TLB 完成转换；影子映射缺失时，由信号处理程序遍历客户机页表并建立映射[12]。BTMMU 则在宿主 MMU 中增加 Dual-TLB，先从内核影子页表重填 GTLB，影子项缺失后再进入非特权异常处理程序[13]。BTMMU 对比实验将 QEMU 的 SoftTLB miss、HSPT 的 segmentation fault 和 BTMMU 的 GTLB invalid exception 归为三类慢路径异常，并指出它们均需遍历客户机页表建立新映射[13]。

因此，硬件加速能够显著降低公共命中路径和异常率，却没有消除剩余映射构造成本。本文以动态主表和 8 项牺牲表表示现代 QEMU 的软件默认机制，以固定 4096 项主表并关闭牺牲表构造容量压力。后者仅是有限硬件转换缓存的受控代理，不复现具体硬件的关联度、替换策略和异常入口延迟；其结果用于评估慢路径优化的潜在空间，而非直接预测 HSPT 或 BTMMU 的整机加速比。

3 慢路径缓存设计

3.1 设计约束

两个缓存遵循相同约束：不改变 CPUTLBEntry 布局，不在生成代码的公共命中路径增加指令；缓存命中后仍调用原有 tlb_set_page_full()，不绕过 RAM、MMIO、只读存储、脏页和观察点处理；所有失效首先采用统一 generation，使旧项立即失效。运行时提供 off、on、probe 和

adaptive 四种模式。probe 执行查询但强制回到原 路径，用于测量查询成本和潜在覆盖率；adaptive 依据运行期复用率启停慢路径缓存。

3.2 大页转换缓存

QEMU 即使识别到 2 MiB 客户机大页，也通常按 4 KiB 粒度向主 SoftTLB 安装表项。随机访问同一大页的不同子页会重复进入 fill 和页表遍历。本文为每个内存管理模式 设置 32 组、4 路，共 128 项的大页转换缓存。缓存项保存客户机虚拟基址、物理基址、完整转换元数据、权限、线性转换范围和 generation。只有目标架构明确给出大于 4 KiB 的线性转换范围时才允许插入。

设虚拟地址为 v ，缓存基址为 v_b 和 p_b ，线性范围为 2^S 。当 v 位于该范围内时，物理地址按式 (1) 重建：

$$p = p_b + (v - v_b), 0 \leq v - v_b < 2^S \quad (1)$$

命中后使用保存的 CPUTLBEntryFull 元数据安装当前 4 KiB 子页。带 PAGE_WRITE_INV 的项不服务写访问，权限不足时也不命中。该设计减少 target fill 和页表遍历，但仍保留主表查询、4 KiB 安装及原有地址空间处理。

实现中必须区分“失效覆盖页大小”和“可线性重建的转换范围”。嵌套翻译为保证失效 正确性可能取两阶段页大小的较大值，而线性映射只能取两阶段范围的较小值。本文新增 lg_translation_size 表达后者，避免以过大范围重建相邻子页的错误物理地址。

3.3 非叶页表缓存

非叶页表缓存为每个虚拟 CPU 设置 64 组、4 路，保存 x86-64 L2—L4 页表遍历的 中间结果。键由 CR3、分页模式、MMU index、PTW index、层级及虚拟地址前缀组成，值包含下一层页表物理地址和累计权限。命中后 walker 从更低层继续，叶页表项仍 重新读取，以保留最终映射及 accessed/dirty 位语义。该实现覆盖四级页表、LA57 和 嵌套页表路径，并在 full、page 和 range TLB invalidation 时推进 generation。

与硬件 MMU cache 不同，软件查询本身包含哈希、标签比较和分支。只有被跳过页表访问的宿主成本高于查询成本时，非叶缓存才能形成净收益。因此该缓存不进入公共 SoftTLB 命中路径，并与大页缓存独立启停。

3.4 自适应低收益旁路

慢路径缓存仍会为每次候选事件增加标签查询和分支。缓存命中率不能精确预测端到端 加速，但在复用不足时继续查询只会增加成本。本文因而为每个虚拟 CPU 的两类缓存 设置独立的三阶段控制器。控制器先观察 2048 次候选查询；若达到收益阈值，则连续 启用 16384 次，否则旁路随后 16384 次候选事件，再重新观察。查询、插入和控制器 均位于主 SoftTLB 与 victim 未命中之后，不进入生成代码的公共命中路径。

对大页缓存，一次命中可避免一次目标架构 fill，窗口内至少每 32 次查询命中 1 次 即视为有效。对非叶页表缓存，命中层级 (l) 可跳过 ($5-l$) 个上层访问，因而按 跳过层数加权，要求平均每 8 次查询至少跳过 1 层。该策略使用软件可直接获得的机制 事件作为低成本代理，不需要宿主性能计数器，也不假定命中与整机收益严格成正比。 固定阈值用于识别明显无复用区间；接近盈亏边界的情况仍须由离线计时判断。

3.5 正确性验证

测试覆盖 4 KiB 与透明大页、随机与连续访问、读写权限、mprotect、页级和范围 失效、嵌套翻译及校验和一致性。mprotect 测试完成 32768 次访问并得到相同校验和 4177920，大页缓存命

中 29893 次且经历 27 次失效；PTW cache 经历 16 次失效。nested 的 base、LP 和 PTW 运行均得到校验和 66846720，64 MiB 映射中有 63488 KiB 经 `/proc/self/smaps` 确认为透明大页。上述 `lg_translation_size` 边界通过嵌套大页回归用例验证。

自适应模式下，最终 `mprotect` 回归执行 1048576 次访问，校验和为 133693440；LP 和 PTW cache 分别经历 30 次和 64 次失效，`refill` 计数关系保持为 0 残差。该结果通过页表遍历、容量快照和 QEMU 版本校验。

4 实验方法

4.1 实验环境

表 2 给出实验环境。优化评价使用无 profile 计数的同一 QEMU binary，客户机磁盘采用 `qcow2 snapshot`。候选配置在每轮内按固定种子随机排序，以减弱温度和后台负载的时间趋势。正式长测将 QEMU 固定在 P-Core logical CPU 7，以 `nice=-20` 运行，将同核 CPU 6/7 的频率请求固定为 2.1 GHz，并临时屏蔽宿主休眠目标；测试结束后恢复宿主设置。桌面文件索引进程在正式样本期间停止。宿主曾在早期 GAPBS 准备阶段进入 `suspend`；受影响测量被排除并按相同配置重跑，原始记录保留用于审计。

表 2 实验环境

项目	配置
QEMU	8.2.9, x86_64-softmmu, -accel tcg,thread=single
宿主处理器	Intel Core i7-1260P, P-Core logical CPU 7, <code>nice=-20</code>
宿主频率	CPU 6/7 使用 <code>performance governor</code> , <code>min=max=2.1 GHz</code>
客户机	Debian 12, Linux 6.1.0-53-cloud-amd64, 1 vCPU
SoftTLB	动态伸缩主表加 8 项 <code>victim</code> ；或每个 MMU mode 固定 4096 项并关闭 <code>victim</code>
客户机内存	一般负载 1 GiB, GAPBS 4 GiB, 微基准 512 MiB
大页	THP= <code>always</code> , 并在相关负载中检查实际覆盖
重复次数	定向微基准 5 次, 八负载筛选 3 次, 429.mcf 两种 SoftTLB 条件各 10 次, 4 KiB 负对照各 30 次

4.2 工作负载与指标

定向微基准在 128 MiB 区域执行 128 轮随机访问，分别使用 2 MiB 透明大页和 4 KiB 普通页；自适应负对照将 4 KiB 随机访问延长到 512 轮，并把 SoftTLB 固定为 64 项，使无大页复用时产生足够多的候选查询。应用集合包括 `sysbench random memory`、`stress-ng tlb-shutdown`、DaCapo 9.12 `avrora`[5]、GAPBS `PageRank scale-20`[7]、嵌套 QEMU/KVM `random-page`，以及 SPEC CPU2006[6] 的 429.mcf、471.omnetpp 和 483.xalancbmk train 输入。

`sysbench` 以 MiB/s 为主指标，`stress-ng` 以 `real-time bogo/s` 为主指标，DaCapo 使用客户机自报时间，nested 使用内层 `ns/access`，其他负载使用测量窗墙钟。性能比统一定义为候选性能除以基线性能；对时间型指标等价于基线中位数除以候选中位数。表中同时报告候选配置的样本变异系数（coefficient of variation, CV）。考虑跨批次时间漂移，429.mcf 和负对照还按 `repetition` 将候选与同轮基线配对，报告配对加速比的中位数、10 万次重采样的 95% bootstrap 区间及快于基线的轮数。每次客户机启动仍可能获得不同的地址空间随机化布局；随机执行顺序不能消除该布局与直接映射表冲突造成的长尾，故异常慢样本保留并在讨论中单独说明。

5 实验结果

5.1 定向微基准

表 3 给出固定 4096 项、关闭 victim 时的定向随机访存结果。大页缓存将透明大页 场景的每次访问中位延迟从 306.698 ns 降至 219.177 ns，获得 1.399 倍加速；组合方案为 1.401 倍，表明 PTW 查询没有额外增益。在 4 KiB 场景中，PTW cache 将延迟从 314.947 ns 降至 295.490 ns，获得 1.066 倍加速；大页缓存因无可复用范围而基本持平。

表 3 定向随机访存微基准（括号内为 CV 和加速比）

客户机页	base/(ns/access)	LP/(ns/access)	PTW/(ns/access)	LP+PTW/(ns/access)
2 MiB THP	306.698 (0.48%)	219.177 (0.48%; 1.399)	311.741 (1.22%; 0.984)	218.849 (0.19%; 1.401)
4 KiB	314.947 (0.28%)	317.663 (0.63%; 0.991)	295.490 (0.70%; 1.066)	295.954 (0.67%; 1.064)

profile 运行显示，透明大页下 LP 将约 419.5 万次 refill 降至约 6.6 万次；4 KiB 下 PTW 的 L2 层命中约 419.5 万次。这说明两种机制分别减少了预期的 fill 和层级 访问，性能变化与机制计数方向一致。

5.2 应用负载

表 4 报告固定 4096 项、关闭 victim 时八个应用负载的主指标。大于 1 表示候选配置 更快，括号内为候选 CV；除 429.mcf 为 10 次重复外，其余负载为 3 次。

表 4 应用负载性能

工作负载	n	base 中位数	LP	PTW	LP+PTW
sysbench	3	159.310 MiB/s	1.002 (0.15%)	1.003 (0.59%)	1.001 (0.08%)
stress-ng	3	346.470 bogo/s	1.025 (1.47%)	1.039 (1.34%)	1.070 (0.99%)
SPEC 429.mcf	10	56.391 s	1.059 (5.39%)	1.025 (4.42%)	1.115 (6.39%)
GAPBS PageRank	3	5.526 s	0.997 (0.74%)	0.977 (3.09%)	0.995 (8.80%)
DaCapo avrora	3	73699 ms	0.946 (3.08%)	0.955 (10.99%)	0.985 (5.37%)
nested hotspot	3	526.092 ns/access	2.244 (4.11%)	0.970 (2.64%)	2.269 (3.80%)
SPEC 471.omnetpp	3	671.088 s	1.066 (0.58%)	1.041 (2.78%)	1.026 (3.10%)
SPEC 483.xalanbmk	3	721.315 s	1.002 (0.38%)	0.993 (2.59%)	0.985 (2.38%)

429.mcf 的 LP 和组合方案按原始中位数分别达到 1.059 倍和 1.115 倍。逐轮配对 加速比的中位数分别为 1.061 和 1.114，95% bootstrap 区间为 [1.057, 1.083] 和 [1.078, 1.141]，且两者均在 10/10 轮快于基线；PTW 的配对中位数为 1.024，区间 为 [1.013, 1.027]，同样为 10/10 轮。独立 profile 中，LP 命中率为 68.55%； 组合配置的 LP 和 PTW 命中率分别为 67.96% 和 99.81%。两种缓存分别复用大页转换 和非叶遍历结果，组合收益高于单一机制，说明该负载同时包含两类可利用的转换局部性。

nested 的 LP 和组合方案在内层随机访问热点达到 2.244 倍和 2.269 倍，但外层测量 窗仅从 16.173 s 降至 15.201 s 和 15.279 s，即 1.064 倍和 1.059 倍。启动、设备 和内层虚拟机的其他工作稀释了热点收益。单独 PTW cache 的热点性能为 0.970 倍， 表明软件查询成本超过其在该配置下跳过层级的收益。

471.omnetpp 的 LP、PTW 和组合方案分别为 1.066 倍、1.041 倍和 1.026 倍，但基线墙钟 CV 为 4.66%，三次重复尚不足以确认小幅差异。483.xalancbmk 基本持平或略有下降。sysbench 和 GAPBS 也未表现出稳定收益，DaCapo 三种配置均慢于基线且 PTW 波动较大。这些结果说明缓存命中率只描述已经越过两级 SoftTLB 的少数事件，不能单独预测端到端收益。

stress-ng 的 bogo/s 在组合配置下提高 7.0%，但该负载使用固定 timeout，外层墙钟基本不变。该结果表示单位时间内完成的 shutdown 工作量增加，而不是程序完成时间缩短，因此不与完成型 workload 的加速比等同解释。

5.3 自适应门控与默认 SoftTLB

为检验有限转换覆盖和 QEMU 默认机制下的差异，表 5 给出 429.mcf 的独立重测。固定容量结果使用桌面索引停止后的 r03—r12；更早两轮原始记录保留，但不并入统计。表中“中位比”由各配置墙钟中位数计算，“配对比”按同轮样本计算，二者因运行顺序和长尾不同而不必相等。

表 5 429.mcf 的自适应重测结果

SoftTLB 配置	方案	n	墙钟中位数/s (CV)	中位比	配对比中位数[95%区间]	胜出轮数
固定 4096、无 victim	base	10	53.697 (5.97 %)	1.000	—	—
固定 4096、无 victim	LP+PTW	10	49.196 (3.49 %)	1.092	1.101[1.065, 1.159]	10/10
固定 4096、无 victim	自适应 LP+PTW	10	48.463 (22.9 1%)	1.108	1.100[1.053, 1.143]	9/10
动态、8 项 victim	base	10	40.590 (3.35 %)	1.000	—	—
动态、8 项 victim	LP	10	40.657 (1.72 %)	0.998	0.995[0.960, 1.053]	5/10
动态、8 项 victim	PTW	10	41.835 (32.9 5%)	0.970	0.975[0.854, 0.995]	1/10
动态、8 项 victim	LP+PTW	10	40.647 (2.80 %)	0.999	0.986[0.971, 1.027]	3/10
动态、8 项 victim	自适应 LP+PTW	10	40.620 (11.2 9%)	0.999	0.998[0.963, 1.037]	5/10

固定容量下，始终启用和自适应组合的配对中位加速均约为 1.100 倍，区间下界均高于 1。自适应配置有一轮 86.929 s 的长尾，因此 CV 明显升高且仅 9/10 轮胜出；系统日志未发现 suspend、热保护、内存不足或 NVMe 错误，现有证据只能把它归为 客户机地址布局、直接映射冲突或未观测宿主扰动的候选原因。默认动态 SoftTLB 加 victim 已先行吸收大部分局部性，四种候选的墙钟中位均与基线相差不足 3%；其中 PTW 出现 91.707 s 长尾且配对区间低于 1，说明无条件查询在该条件下可能形成负收益。

独立 profile 表明，固定容量自适应组合在约 6166 万次 LP 查询中命中 3479 万次，控制器进入 3732 个启用窗和 253 个旁路窗；PTW 的约 3109 万次查询中命中约 3098 万次，未进入旁路。默认动态配置中，victim 先挽回约 197 万次请求；LP 仅旁路 4 个窗口，PTW 未旁路，符合该负载仍具有慢路径局部性的机制判断。门控并不会因端到端噪声自动关闭一个机制，它只识别机制事件层面的明显低收益。

表 6 给出无大页复用的 4 KiB 负对照。三组中央趋势无可辨差异，但始终启用 LP 将 CV 从 5.90% 提高到 12.70%，自适应模式恢复到 6.02%。机制运行中，控制器旁路 738314/861993，即 85.7% 的候选事件，说明它确实停止了多数低收益查询。

表 6 4 KiB 随机负对照（每组 30 次）

方案	中位延迟/(ns/ access)	CV	中位比	配对比中位数 [95%区间]	胜出轮数
base	336.509	5.90%	1.000	—	—
LP	337.153	12.70%	0.998	0.999[0.991, 1.006]	14/30
自适应 LP	336.972	6.02%	0.999	0.995[0.941, 1.058]	14/30

5.4 收益边界

设基线时间中可被慢路径缓存优化的比例为 f ，该部分加速比为 s ，则端到端上限为

$$S_{total} = 1 / [(1 - f) + f/s] \quad (2)$$

现代 QEMU 的动态主表和牺牲表已过滤大多数转换请求。当 refill 低于访存次数的 0.5% 时，即使单次 fill 明显变快， f 仍可能过小。另一方面，缓存查询、4 KiB 表项安装和地址空间解析仍然存在，较高命中率也可能被这些固定成本抵消。因此，429.mcf 的收益来自较高 refill 比例及两类局部性共同作用，而 GAPBS、DaCapo 和 483.xalancbmk 的结果不能通过简单扩大缓存容量改善。

6 讨论

本文的创新点在于把大页转换复用和非叶页表复用放入现代 QEMU 的 miss-only 路径，以不改变公共生成代码为约束，并显式区分失效覆盖范围与线性转换范围。相比直接扩大主 SoftTLB 或增加关联度，该方案不会使每次访存承担额外比较；相比绕过 QEMU 地址空间安装过程，该方案保留了 MMIO、权限和观察点语义。两种机制可独立启用，适合按负载的 refill、页表层级和失效频率进行选择。

固定 4096 项主表并关闭牺牲表是一种受控压力配置，用于减少动态表容量差异并暴露慢路径机会，不代表 QEMU 默认配置，更不能直接等同于某种硬件 TLB。实验仅覆盖 x86-64、单虚拟 CPU 和 single-thread TCG，宿主也只是一台 i7-1260P；同一物理核的 SMT sibling 在线，八负载筛选中每个配置只有三次重复，因此小幅变化应视为趋势。named SoftMMU 不包含内联命中查询，其时间占比是慢路径下界。

当前门控阈值是依据一次 fill 或所跳过层级数设定的启发式代理，状态按虚拟 CPU 聚合，尚未区分地址空间、MMU mode 或执行阶段。它能够旁路明显无复用区间，却不能直接观测宿主缓存、分支预测和调度开销，因而不保证消除端到端回退。固定容量与默认配置均保留了无明确系统故障证据的长尾，提示客户机 ASLR、缓存索引冲突及宿主未观测扰动仍需通过地址布局控制、SMT sibling 离线和更多机器复验。后续还应在更大图分析、服务器负载和多虚拟 CPU 条件下调节采样窗、阈值及状态粒度。

7 相关工作

Tong 等系统研究 QEMU 1.7.0 的访存模拟，比较 SoftTLB 容量、牺牲表、关联度、大页和 refill 优化[1]；Hong 等进一步分析 QEMU 2.2.0 的控制转移与内存虚拟化，强调动态容量和大页失效的共同影响[2]。本文针对已具有动态主表和牺牲表的 QEMU 8.2.9，重点处理两级表均未命中的剩余慢路径。

Barr 等提出硬件 translation cache，通过保存部分转换跳过页表层级[3]。Victima 利用末级缓存扩大地址转换覆盖范围[8]，Utopia 通过受限与灵活映射结合降低转换成本 [9]，Ninja 面向嵌套地

址转换提供硬件加速[10]。这些工作证明转换复用和减少页表 层级的价值；本文研究相同局部性在软件全系统模拟器中的实现边界，并把软件哈希、 分支、失效和 QEMU 地址空间语义纳入设计。

HSPT 通过宿主页表和硬件 TLB 直接服务客户机访问，并以信号处理维护影子映射[12]；BTMMU 通过 Dual-TLB、硬件异常入口和内核影子页表支持更广泛的跨 ISA 地址空间与 页大小组合[13]。二者的公共路径不同于 QEMU SoftTLB，但影子项缺失后的慢路径仍需 执行客户机页表遍历和映射建立。本文补充其未重点研究的慢路径内部缓存优化，并以 固定容量实验估计硬件命中受限时的潜在收益。

8 结论

本文面向现代全系统动态二进制翻译器中低频、高代价的地址转换慢路径，设计大页转换 缓存、非叶页表缓存及在线收益率门控。三者只在主 SoftTLB 与 victim 均未命中后 工作，不增加生成代码公共命中路径的指令；缓存命中仍经原有 4 KiB 表项安装，并以 独立线性转换范围和统一 generation 保持嵌套翻译及失效语义。分离式测量显示，应用 data refill 虽低于访存次数的 0.5%，可命名慢路径仍可占宿主周期的 16.84%。固定 4096 项并关闭 victim 时，定向大页和 4 KiB 微基准分别加速 1.399 倍和 1.066 倍， 429.mcf 的自适应组合取得 1.100 倍配对中位加速，95% 区间为 [1.053, 1.143]；默认动态配置下则为 0.998 倍，未显示可辨收益。无大页复用的负对照中，门控旁路 85.7% 的候选查询，并把波动恢复至接近基线，但长尾仍存在。结果说明，有限转换覆盖 后的剩余慢路径可保留显著局部性，适合 HSPT、BTMMU 等硬件辅助环境在映射缺失后 复用；然而硬件异常入口、关联度和替换策略并未被本实验复现，不能据固定容量结果 直接预测其整机收益。自适应门控能够压低明显低收益区间的附加开销，而不能替代跨 布局、跨宿主和多虚拟 CPU 的稳健性验证。由此，模拟器优化的评价必须同时报告机制 覆盖率、端到端效应及异常样本，不能用局部命中率替代系统性能证据。

参考文献

- [1] TONG X, KOJU T, KAWAHITO M, et al. Optimizing memory translation emulation in full system emulators[J]. ACM Transactions on Architecture and Code Optimization, 2015, 11(4): 1-24.
- [2] HONG D Y, HSU C C, CHOU C Y, et al. Optimizing control transfer and memory virtualization in full system emulators[J]. ACM Transactions on Architecture and Code Optimization, 2015, 12(4): 1-24.
- [3] BARR T W, COX A L, RIXNER S. Translation caching: skip, don't walk the page table[C]//Proceedings of the 37th Annual International Symposium on Computer Architecture. Saint-Malo: ACM, 2010: 48-59.
- [4] BELLARD F. QEMU, a fast and portable dynamic translator[C]//Proceedings of the USENIX Annual Technical Conference. Anaheim: USENIX Association, 2005: 41-46.
- [5] BLACKBURN S M, GARNER R, HOFFMANN C, et al. The DaCapo benchmarks: Java benchmarking development and analysis[C]//Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications. Portland: ACM, 2006: 169-190.
- [6] HENNING J L. SPEC CPU2006 benchmark descriptions[J]. ACM SIGARCH Computer Architecture News, 2006, 34(4): 1-17.
- [7] BEAMER S, ASANOVIC K, PATTERSON D. The GAP benchmark suite[EB/OL]. (2015)[2026-09-12]. <https://github.com/sbeamer/gapbs>.

- [8] KANELLOPOULOS K, NAM H C, BOSTANCI N, et al. Victima: drastically increasing address translation reach by leveraging underutilized cache resources[C]//Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. Toronto: ACM, 2023: 1178-1195.
- [9] KANELLOPOULOS K, BERA R, STOJILJKOVIC K, et al. Utopia: fast and efficient address translation via hybrid restrictive and flexible virtual-to-physical address mappings[C]//Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture. Toronto: ACM, 2023: 1196-1212.
- [10] ZHAO L Y, WANG Z W, LIU F X, et al. Ninja: a hardware assisted system for accelerating nested address translation[C]//Proceedings of the 42nd IEEE International Conference on Computer Design. Milan: IEEE, 2024: 426-433.
- [11] LINDSAY N, BHATTACHARJEE A. Understanding address translation scaling behaviours using hardware performance counters[C]//Proceedings of the IEEE International Symposium on Workload Characterization. Vancouver: IEEE, 2024: 236-246.
- [12] WANG Z, LI J J, WU C G, et al. HSPT: practical implementation and efficient management of embedded shadow page tables for cross-ISA system virtual machines[C]//Proceedings of the 11th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. Istanbul: ACM, 2015: 53-64.
- [13] HUANG K, ZHANG F X, LI C, et al. BTMMU: an efficient and versatile cross-ISA memory virtualization[C]//Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments. Virtual Event, USA: ACM, 2021: 71-83.